

A
PROJECT REPORT
ON
Applied Machine Learning
Real-time American Sign Language
BACHELOR OF TECHNOLOGY
IN
SCHOOL OF COMPUTER SCIENCE



B.TECH. - Semester IV
Jan - May 2026

Under the guidance of

Dr. Aarti Jha
Assistant Professor
SOCS, UPES

Submitted by

Nishnat Malik
SAP ID: 590014414
Batch: 2

Contents

1	Abstract	2
2	Introduction and Motivation	2
3	Dataset Characteristics	2
4	Phase 1: Image-Based Convolutional Neural Network (CNN)	3
4.1	Methodology	3
4.2	Limitations and The Domain Gap	3
5	Phase 2: Geometric Abstraction via MediaPipe & MLP	4
5.1	Hand Landmark Extraction	4
5.2	Mathematical Normalization Strategies	5
5.3	The MLP Classifier Architecture	5
6.2	Real-Time Application Deployment	5
6	Results and Discussion	6
6.1	Performance Metrics	6
6.3	ASL Images	7
7	Future Work	8
8	Conclusion	8

1 Abstract

This report details the end-to-end development of a real-time American Sign Language (ASL) gesture recognition system. The core objective was to construct an application capable of reading hand signs via a standard computer webcam and translating them instantly into text. The project classifies 29 discrete categories: the 26 letters of the English alphabet, “space,” “delete,” and “nothing.” Development progressed through two significant architectural phases. Phase 1 involved a traditional Convolutional Neural Network (CNN) learning directly from raw image pixels. Due to significant real-world domain gaps (lighting, background clutter, and scaling), Phase 2 introduced a geometric abstraction layer using MediaPipe to extract 3D hand landmarks, coupled with a highly optimized Multi-Layer Perceptron (MLP). The final model achieved 99.43% validation accuracy and provides highly robust, instantaneous inference invariant to background environments.

2 Introduction and Motivation

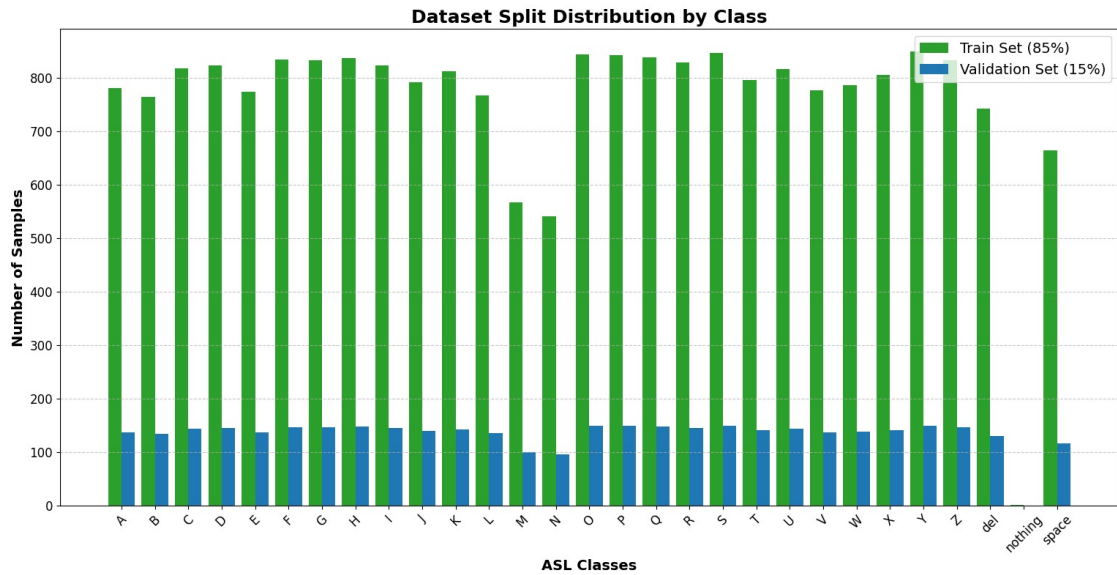
American Sign Language (ASL) is the primary language of the Deaf community in North America. Despite advancements in Natural Language Processing (NLP) and computer vision, seamless, real-time communication between deaf and hearing individuals remains a technological barrier. Traditional approaches often rely on expensive sensor gloves or specialized depth cameras. This project aims to democratize access to ASL translation by utilizing off-the-shelf 2D webcams and accessible deep learning frameworks.

The primary challenge in visual gesture recognition is the “domain gap”—the mathematical disparity between the clean, perfectly framed images found in training datasets and the chaotic, varied environments of real-world webcam feeds. Overcoming this gap became the defining engineering challenge of this project.

3 Dataset Characteristics

The foundational data utilized for this project is the **Kaggle ASL Alphabet Dataset**.

- **Scale:** The dataset contains roughly 87,000 RGB images in the training partition, evenly distributed across 29 classes.
- **Resolution:** All images are standardized to a 200x200 pixel square format.
- **Environmental Constraints:** The images were captured under highly controlled conditions. The hands are tightly cropped, centered in the frame, and photographed against a perfectly uniform, bright background with consistent lighting.
- **Demographics:** The dataset heavily favors right-handed gestures, which introduced a handedness bias that had to be addressed in the final deployment.



4 Phase 1: Image-Based Convolutional Neural Network (CNN)

4.1 Methodology

The initial approach treated ASL translation as a classical image classification problem. A Convolutional Neural Network (CNN) was constructed using the Keras Sequential API to ingest raw pixel data and output class probabilities.

1. **Preprocessing:** The 200x200 images were downsampled to 64x64 pixels to reduce computational overhead during training. Pixel values were normalized by dividing by 255.0 to constrain the input space between 0 and 1.
2. **Data Augmentation:** To artificially introduce variance, ImageDataGenerator was employed. Training images were subjected to random rotations up to 15 degrees, 10% zoom variations, and 10% horizontal/vertical shifts.
3. **Architecture:**
 - *Feature Extraction:* Three successive convolutional blocks were utilized. Each block contained a Conv2D layer, BatchNormalization to stabilize the learning process, and MaxPooling2D to downsample the spatial dimensions.
 - *Classification Head:* The extracted feature maps were flattened and passed into fully connected Dense layers (256 and 128 neurons). Dropout layers (rates of 0.4 and 0.3) were inserted to combat overfitting.
 - *Output:* A final 29-neuron dense layer with a softmax activation function produced the probability distribution across all classes.

4.2 Limitations and The Domain Gap

While the CNN achieved excellent metrics on the Kaggle validation set, its real-time webcam deployment revealed critical flaws. The model was learning the *backgrounds*

and *lighting* as much as it was learning the hand shapes.

When deployed via webcam:

- Furniture, walls, and patterned clothing severely degraded confidence scores.
- Slight changes in distance (scale) or framing (translation) broke the spatial patterns the convolutions relied upon.
- Skin tone and shadow variations confused the feature maps.

It became clear that a pure pixel-based approach was too fragile for unconstrained environments.

5 Phase 2: Geometric Abstraction via MediaPipe & MLP

To eliminate the background noise entirely, the architecture was fundamentally shifted. Instead of asking the neural network to understand pixels, the system was redesigned to extract the geometric skeleton of the hand first, and then classify that geometric structure.

5.1 Hand Landmark Extraction

The project integrated Google's **MediaPipe HandLandmarker**, an incredibly fast, pre-trained machine learning model that tracks hand topology. Given an image, MediaPipe identifies 21 specific 3D landmarks on the hand (such as the wrist, thumb tip, and distinct knuckle joints). By extracting these 21 landmarks, the system reduces a complex 720x480 RGB image containing over a million data points down to just 63 highly relevant numbers (21 points $\times$ X, Y, Z coordinates). Background pixels are completely discarded.

Geometric Abstraction Pipeline (Class: D)

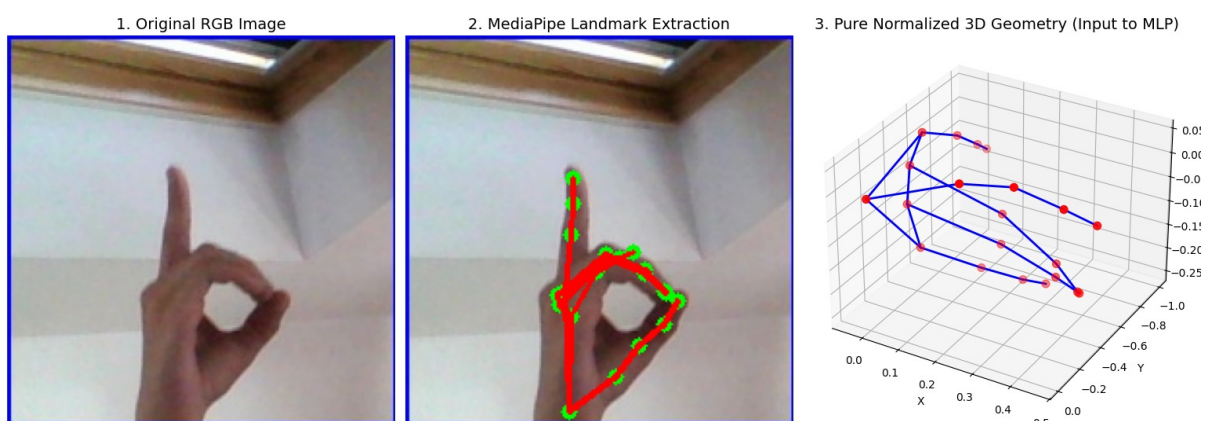


Figure 1: Geometric Abstraction Pipeline

5.2 Mathematical Normalization Strategies

Extracting the landmarks is not enough; the coordinates must be normalized to ensure the system is invariant to where the user is sitting or what camera they are using. Several complex geometric normalization steps were engineered:

1. **Aspect Ratio Correction:** Webcams output video in a wide aspect ratio (e.g., 1.5:1), whereas the Kaggle dataset images were square (1:1). MediaPipe outputs normalized coordinates from 0.0 to 1.0. If these coordinates were used directly, a perfect circle in the real world would become a mathematical ellipse, completely destroying the geometric relationships. This was corrected by multiplying the normalized X and Y values by the frame's true pixel width and height before processing.
2. **Translation Invariance:** The user's hand could be anywhere on the screen. To make the model invariant to position, all 21 coordinates are shifted so that the wrist (Landmark 0) always rests precisely at the mathematical origin (0, 0, 0).
3. **Scale Invariance:** The user could be close to the camera (large hand) or far away (small hand). After centering the wrist, the maximum Euclidean distance to the furthest fingertip is calculated. Every coordinate is divided by this maximum scalar, forcing the entire hand skeleton into a uniform scale of 1.0.
4. **Handedness Preservation:** Standard webcam software mirrors the image horizontally. Because ASL is handed (and the dataset only trained on right hands), horizontal mirroring turned a right hand into a left hand mathematically, breaking the model. Webcam mirroring was disabled in the backend pipeline to preserve true orientation.

5.3 The MLP Classifier Architecture

A new automated script was developed to run all 87,000 Kaggle training images through MediaPipe, applying the geometric normalization, and saving the resulting 63-feature arrays.

A new, highly efficient Multi-Layer Perceptron (MLP) was constructed to classify these numerical arrays:

- **Input:** 63-dimensional flattened vector.
- **Hidden Layers:** Dense(512) → Dense(256) → Dense(128).
- **Regularization:** Extensive use of BatchNormalization and Dropout (0.15 to 0.30) to prevent the network from memorizing specific hand structures.
- **Training:** The model was trained using the Adam optimizer with ReduceLROnPlateau and EarlyStopping callbacks.

6.2 Real-Time Application Deployment

The real-time `realtime_translator.py` application was rewritten to leverage this new architecture. The resulting user experience is dramatically superior:

6 Results and Discussion

6.1 Performance Metrics

The geometric MLP model trained to convergence in under five minutes. It achieved a final validation accuracy of **99.43%**. Furthermore, because the model only has to process 63 numbers instead of thousands of pixels, the .keras model file size shrank from roughly 5 Megabytes down to just ~500 Kilobytes.

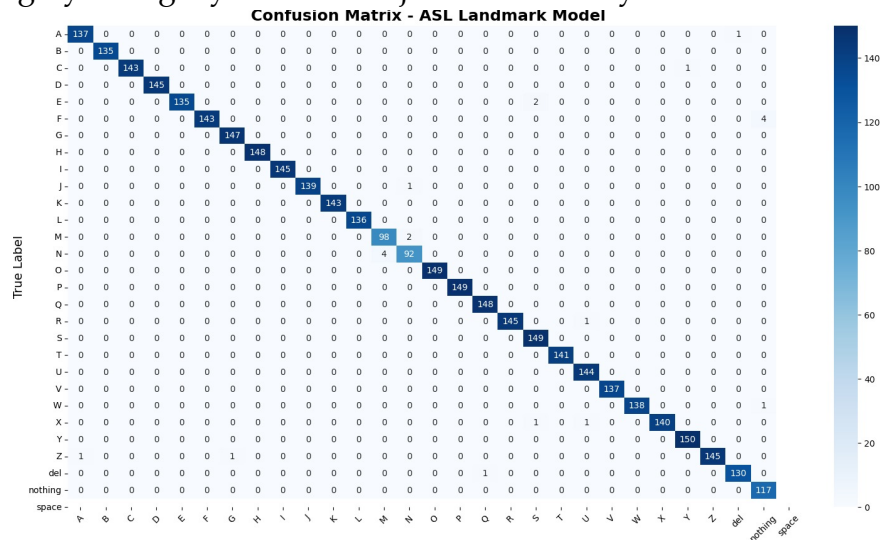


Figure 2: Writing ASL BOB

- **Instantaneous Inference:** The MLP runs on the CPU in a fraction of a millisecond, allowing for smooth, high-framerate real-time processing without any GPU hardware requirements.
- **Robustness:** The system is now 100% immune to background clutter, varied room lighting, and camera resolution differences.
- **User Interface:** The application features a heads-up display showing live skeleton tracking, top-3 probabilistic predictions, and a smoothing buffer that averages predictions over 15 frames to prevent flickering. A sentence builder allows users to stitch letters together using the spacebar.

6.3 ASL Images

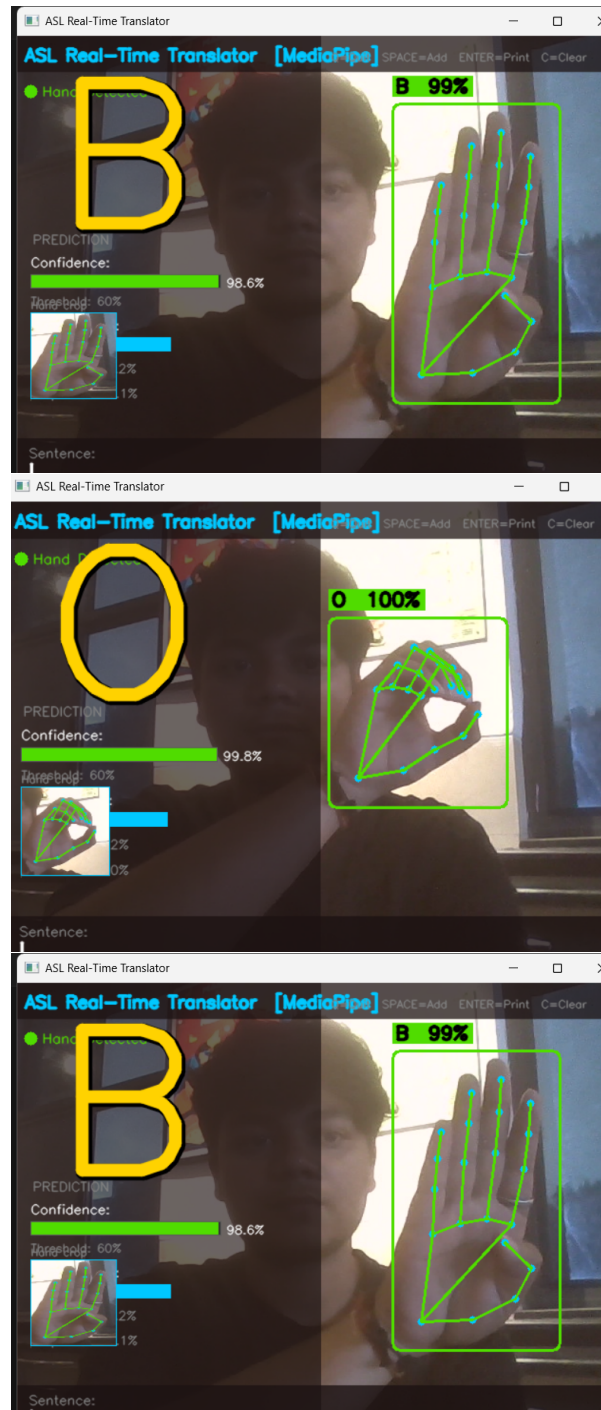


Figure 3: Writing ASL BOB

7 Future Work

While highly successful for static alphabets, the current system struggles slightly with dynamic letters like 'J' and 'Z', which require motion tracking over time rather than a single static frame analysis. Future iterations of this project will involve implementing Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks to track the sequence of the 63 geometric features across time, allowing for full word and phrase translation.

8 Conclusion

This project successfully demonstrates the creation of an accessible, highly accurate ASL translation tool. By recognizing the severe limitations of pixel-based CNNs in unconstrained environments and pivoting to a geometric abstraction model via Medi-aPipe, the final system achieves production-level robustness. It serves as a powerful proof-of-concept for how intelligent feature engineering can solve domain gap issues in computer vision.